



“THEME ARTICLE”, “FEATURE ARTICLE”, or “COLUMN” goes here: The theme topic or column/department name goes after the colon.

Heterogeneous Hierarchical Workflow Composition

Orcun Yildiz

Argonne National Laboratory

Jorge Ejarque

Barcelona Supercomputing Center

Henry Chan

Argonne National Laboratory

Subramanian

Sankaranarayanan

Argonne National Laboratory

Rosa M. Badia

Barcelona Supercomputing Center

Spanish National Research Council

Tom Peterka

Argonne National Laboratory

Workflow systems promise scientists an automated end-to-end path from hypothesis to discovery. However, it is impractical to expect any single workflow system to deliver such a wide range of capabilities. A more practical solution is to compose the end-to-end workflow from more than one system. With this goal in mind, the integration between distributed and in situ workflows is explored, where the result is a hierarchical heterogeneous workflow composed of subworkflows; with different levels of the hierarchy using different programming, execution, and data models. Materials science use cases demonstrate the advantages of such heterogeneous hierarchical workflow composition.

Scientific computing consists of multiple related computational tasks. For instance, the detection of high-turbulence potentially destructive features in plasma physics simulations requires the in situ coupling of two simulation codes, a compression tool, a feature detection algorithm, and a visualization library.¹ Such complex workflows require significant effort from scientists to manage the scheduling and data exchange among those tasks. To aid in automating this process, scientific workflow frameworks allow scientists to define the dependencies and data exchanges among connected tasks instead of managing those manually, potentially resulting in increased scientific productivity.

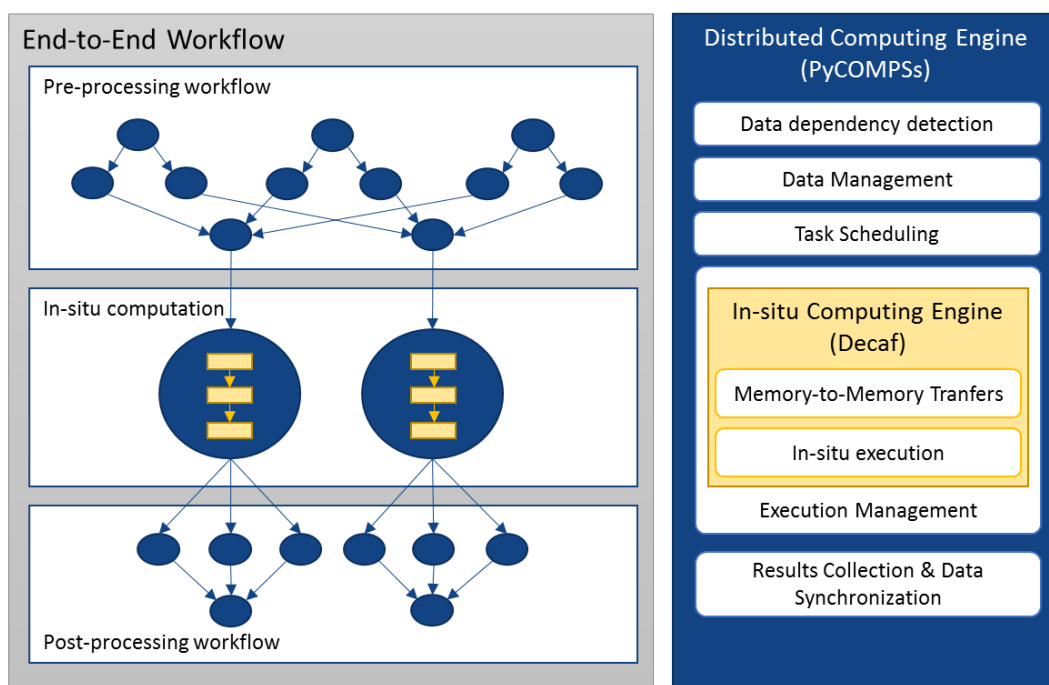


Figure 1. Overview of the proposed approach where in situ workflows are integrated as tasks in an end-to-end distributed workflow. The result is a heterogeneous workflow with different levels of hierarchy using different programming, execution and data models.

There are two main classes of scientific workflows that we consider in this work: in situ and distributed workflows. In situ workflows run within a single HPC system, and the data exchange is done through memory or the supercomputer interconnect during the same scheduled execution of a job.²⁻⁵ Distributed workflows run across several independent systems in a wide area such as grids and clouds, where the tasks exchange data through files.⁶⁻⁹

These workflow solutions are optimized to run on their respective scientific computing environments, but many of today's scientific applications include both in situ and distributed components; for example, initial data processing in situ may be followed by offloading to wide-area systems for further processing.¹⁰ Unfortunately, in situ and distributed workflow systems lack interoperability with each other. Hence, today's workflow solutions still cannot support an automated end-to-end path from hypothesis to discovery.

In this work, we investigate the integration between distributed and in situ HPC workflows. Figure 1 shows our approach where we employ in situ workflows as tasks of a high-level end-to-end workflow that is managed by a distributed workflow tool. The result is a heterogeneous hierarchical workflow that is designed to integrate two completely different workflow programming models: task-based distributed workflows with in situ HPC workflows. This integration requires providing an interface to describe in situ computations in a distributed workflow without adding extra complexity and providing efficient execution of the in situ computations without interfering with the other tasks of the distributed workflow.

We evaluated our approach by performing experiments in the field of materials science. In particular, the in situ workflow consists of the molecular dynamics simulation coupled to a parallel in situ feature detector that selects nucleated molecule clusters during crystallization. Meanwhile, the distributed workflow launches an ensemble of in situ workflows with different initial conditions, saves results from experiments that successfully nucleated, and collects snapshots from those ensemble members into an animation.

Our results reveal that heterogeneous workflow integration can benefit both distributed and in situ workflow systems. We believe that the insights and lessons learned from our integration will increase understanding and motivate further research into heterogeneous workflow composition.

In Situ Workflows

In situ workflows address the mismatch between I/O and computing capabilities of supercomputers by allowing users to perform data analysis at simulation time. This in turn minimizes the I/O time and shortens the overall time to discovery.

Several in situ workflow solutions have been proposed. Decaf⁵ is a middleware for building and executing in situ workflows, allowing parallel communication of coupled tasks by creating communication channels over HPC interconnects through MPI. It provides a Python API to describe the workflow graph. [Decaf does not impose any constraints on this graph topology and can manage graphs with cycles.](#)

Some widely used visualization tools also extended their APIs to expose the simulation data in situ to their visualization engines. Examples of this approach include VisIt's Libsim² and ParaView's Catalyst³ libraries.

We have also witnessed the extension of I/O libraries to support in situ analysis. ADIOS,¹² originally designed for I/O staging, can now couple analysis and visualization applications together with simulations through an I/O interface.⁴ Damaris¹³ is a I/O middleware that supports in situ data processing and visualization using dedicated cores or nodes of the simulation platform.

Distributed Workflows

Workflow management systems for distributed computing became popular a decade ago as a tool to deploy scientific workflows in grid computing environments. Several survey papers¹⁴⁻¹⁵ present taxonomies of representative systems from that period.

Distributed workflow systems execute in different ways. For example, Taverna⁶ orchestrates previously deployed web services. This model offers some advantages in decoupled environments and fits well in cloud computing, but its performance overheads are not well suited for HPC workflows. Most systems are oriented to compose external binaries and tools (e.g., Galaxy⁷), but only a few can directly invoke methods described in programming languages (e.g., PyCOMPSs/COMPSs⁸).

The use of parallelism in the tasks of the workflow has been limited and until recently, most workflow systems executed each task sequentially. There are some cases where tasks invoke parallel MPI applications, as in the case of the fusion community, which uses Kepler¹⁶ as a workflow management system. Another case is the climate community, which uses tailored workflow management systems based on simple tagged scripts (Cylc¹⁷ or Autosubmit¹⁸). Their workflows are typically composed of several invocations of simulations, programmed with MPI, which are offloaded for execution to the job scheduler of a cluster or supercomputer.

Hybrid Workflows

End-to-end scientific applications can require different capabilities of several execution environments (e.g., HPC and distributed systems) during their lifetime. For instance, the Cybershake earthquake hazard model contains MPI jobs, which in turn generate distributed jobs that will run on grids.¹⁹ To achieve this, Cybershake uses Pegasus⁹ as a [workflow management system](#). However, Pegasus is designed primarily for distributed workflows, and cannot provide all the advantages of in situ workflows [such as efficient data transfer through shared memory or HPC interconnect instead of files.](#)

To the best of our knowledge, there are no software frameworks available that can provide all the capabilities of both in situ and distributed environments. To fulfill the requirements of complex scientific applications, we investigate whether hybrid heterogeneous workflow combinations are

a viable way to span different execution environments. Next, we present one such integration towards this goal.

HETEROGENEOUS WORKFLOW INTEGRATION

Here, we present our methodology to integrate distributed workflows with in situ workflows. With this integration, we can automate end-to-end path from hypothesis to discovery by using these different workflows for what they do best. Our approach consists of employing one or more in situ subworkflows as tasks of a high-level end-to-end workflow, thereby integrating these in situ computations with the other pre/post processing phases. The end-to-end workflow is managed by a distributed workflow tool that performs data dependency analysis, schedules the tasks on available resources, performs the required data transfers, manages the task execution, and retrieves the results of computations. In situ subworkflows can be described inside this distributed workflow by indicating their input/output data and computational requirements. This is sufficient to enable the distributed workflow tool to manage entire in situ workflows as single tasks in terms of data dependencies, scheduling, and data transfers. Regarding the execution of these in situ tasks, the distributed workflow tool configures them to use the assigned resources and provides the location of the input and the output data. Next, the distributed workflow tool starts the execution of the in situ computation and monitors it until completion. Then, the distributed workflow tool retrieves the results and makes them available to other tasks of the workflow.

We implemented our approach using PyCOMPSs and Decaf as the distributed and in situ workflow tools, respectively. PyCOMPSs enables users to program task-based parallel workflows as Python scripts by simply annotating Python methods with a *task* decorator. Inside this decorator, users annotate the direction of the method arguments as input or output. For instance, if the task only reads data, this is indicated as IN, or if the task also modifies this data, this is indicated as INOUT. Besides the task decorator, users can also specify constraints for a task (e.g., amount of memory, number of CPU cores) with the *constraint* decorator. With this information, the PyCOMPSs runtime can automatically detect data dependencies between the annotated tasks and build a Directed Acyclic Graph (DAG), where a node in the DAG represents a task execution, and edges represent data dependencies. According to this DAG and the defined constraints, task executions are scheduled and remotely executed on the available resources automatically.

A PyCOMPSs application is executed in a master-worker fashion, where the main code of the application is executed on the master node, and tasks are remotely executed on worker nodes. On the other hand, Decaf is a parallel dataflow engine designed to perform efficient in situ computations over HPC interconnects through MPI. Decaf has a Python API, where users can define the different nodes and edges of a dataflow and the number of processes assigned to each dataflow entity. Once the dataflow is defined, it is executed as a Multiple-Program-Multiple-Data (MPMD) MPI application.

```

1  @constraint(ComputingUnits=num_cores)
2  @decaf(dfScript="decaf-config-script.py")
3  @task(infile=FILE_IN, outfile=FILE_OUT, returns=int)
4  def decaf_task_method(infile, outfile):
5      pass
6
7  @constraint(ComputingUnits=num_cores)
8  @task(outfile=FILE_OUT) #in_param input object as default
9  def normal_task(in_param, outfile):
10     ...
11     if __name__ == "__main__":
12         normal_task(param, file1)
13         decaf_task_method(file1, file2)
```

Figure 2. Decaf and normal task description and invocation in PyCOMPSs. Decaf tasks are described similarly to normal tasks.

To integrate a Decaf execution in a PyCOMPSs workflow, we have extended the PyCOMPSs syntax with a *decaf* decorator, which is used to identify and describe the task as an in situ Decaf computation. Figure 2 shows an example of a simple usage of the *decaf* decorator. Users need to define a method without the implementation details, which has the same arguments as the Decaf execution. In the *task* and *constraint* decorators, users state the direction of the parameters and the resource requirements like a normal PyCOMPSs task. In the *decaf* decorator, users indicate that the defined task is a Decaf dataflow whose description is provided in the *dfScript* property. Once the Decaf task is defined, users can invoke it as a regular Python method call and combine with other PyCOMPSs tasks as shown in Figure 2 (lines 12 and 13).

When the PyCOMPSs runtime detects a Decaf task invocation, it analyses the data dependencies of the new node in the DAG. Once the task has satisfied its data dependencies, it is scheduled on the available resources according to the defined constraints. Then, PyCOMPSs transfers the required data to the selected computing nodes and sends the command to proceed with the task execution. PyCOMPSs executes the *decaf-config-script.py* script with the following arguments: the number of MPI processes to spawn (the *ComputingUnits* specified in the *constraint* decorator), the MPI hostfile, and the arguments of the task. The *decaf-config-script.py* script generates the Decaf command to execute the workflow as a MPMD MPI execution. This command is automatically executed by the PyCOMPSs worker runtime, which monitors the execution state. Once the Decaf execution finishes, the PyCOMPSs master runtime is notified in order to proceed with the remaining tasks in the workflow graph that depend on the results of this Decaf execution.

Science Use Case

The materials science problem we study using our PyCOMPSs-Decaf workflow system is nucleation as a material cools and crystallizes. In particular, we study water freezing, but the same workflow applies to nucleation in many other material systems.

Application Description

Understanding the mechanisms and kinetics of crystallization events is key to understanding a wide range of natural and technological systems. Nucleation is a stochastic event that requires a large number of statistics to elucidate its kinetics. It is difficult to capture nucleation in simulations, especially during the early stages when only a few atoms have crystallized.

The way scientists simulate nucleation is either to run one single large simulation or many instances of small simulations. However, both scenarios have their drawbacks. For large simulations, massive computing power is needed, prohibiting most scientists from conducting such computations. On the other hand, it is very difficult to observe nucleation in small simulation systems. To increase their chances, scientists employ advanced sampling techniques by performing many instances of crystallization simulations. Given the stochastic nature of the nucleation process, nucleation can be observed in only a few of these instances. Hence, this is a tedious process for scientists—especially without the aid of workflow systems—where they need to run and analyze many simulation instances hoping to achieve the results they need with only few of them. We demonstrate the applicability of our heterogeneous workflow composition for both scenarios: one large molecular system and many small systems.

Diamond Structure Identification

The initial pre-processing of data includes an identification of different phases of ice—hexagonal, cubic, and amorphous/liquid—using a diamond structure identification algorithm implemented in the OVITO visualization software.²⁰ Briefly, the classifier labels crystalline atoms based on the topological relationship between the nearby first shell and second shell neighbors. Unlabeled atoms, including atoms in the grain boundaries, are considered amorphous.

We developed a parallel version of the diamond structure identification algorithm and coupled it in situ to the LAMMPS¹¹ molecular dynamics model using Decaf. This allowed us to avoid the potential I/O bottleneck associated with writing/reading all atoms to/from storage; instead detecting diamond-shaped structures in situ alongside the molecular dynamics simulations. To further improve the I/O performance, we only stored time steps after a minimum number of atoms (1/8 of the total number of atoms) crystallized. Moreover, within a time step we only stored the crystallized atoms.

To find out the optimal number of processes for the parallel diamond structure identification algorithm, we performed a scalability test where we used a LAMMPS input file consisting of 2,000,000 atoms. We used up to 128 MPI processes and achieved the optimal performance when using 16 processes. The reason is that each subdomain in the parallel decomposition includes the nearest first and second neighbors of any atom in the subdomain. As the subdomains shrink in size with increasing number of processes, the size of this ghost region dominates the computation, limiting scalability.

EXPERIMENTAL METHODOLOGY

Platform Description

Our experiments were conducted using the MareNostrum 4 supercomputer, which has 3,456 computing nodes, each with 2 Intel Xeon Platinum 8160 CPUs with 24 cores each at 2.1 GHz, and 2GB or 8 GB of DDR4 RAM memory per core. All nodes are connected to each other by a 100Gb Intel Omni-Path interconnection network and a 10GB Ethernet network.

Experimental Deployment

We study the nucleation problem with two different simulation sizes: the large simulation is a molecular model of a box of water composed of 2,000,000 atoms, and the small simulation contains 4,000 atoms. For the large simulation, PyCOMPSs launched a single instance of a Decaf subworkflow, which used 2000 processes for LAMMPS with a temperature value of 210K, and 16 processes for the diamond structure identification algorithm. For the small simulation, many instances of LAMMPS crystallization simulations are performed. In this regime, PyCOMPSs launched multiple instances of a Decaf subworkflow until there is a nucleation event. We used three different undercoolings (210K, 220K, and 230K) with different random seed values, each instance using 44 processes for LAMMPS and 4 processes for the parallel diamond structure detector.

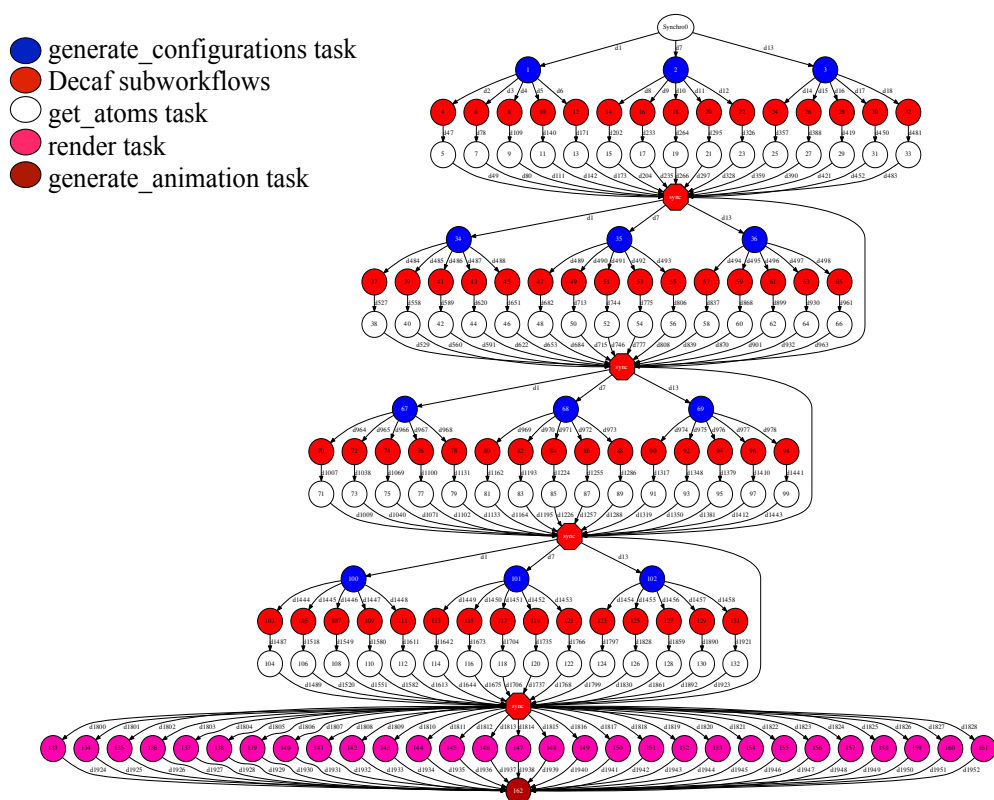


Figure 3. Hierarchical workflow graph dynamically generated by PyCOMPSs depending on whether there is a successful nucleation event. Blue nodes generate a configuration file with different initial conditions (seed variable, temperature) for each simulation instance; red nodes represent the Decaf subworkflows; white nodes fetch the output files from these subworkflows; pink nodes are *render* tasks and the dark red node combines the rendered frames into an animation.

Figure 3 shows the hierarchical workflow graph for the ensemble of such simulations. Blue nodes are the *generate configurations* tasks that generate a configuration file with different initial conditions (seed variable, temperature) for each simulation instance. Red nodes represent the Decaf subworkflows. White nodes are *get atoms* tasks that fetch the output files from these subworkflows. Pink nodes are *render* tasks, and the dark red node is the *generate animation* task, that combines the rendered frames into an animation.

The PyCOMPSs task graph is dynamic and varies depending on whether nucleation succeeds. When nucleation does not occur, PyCOMPSs discards the output files produced by Decaf subworkflows and continues spawning more of these subworkflows, as we can see during the first four iterations in Figure 3. When it detects a successful nucleation, the workflow executes the *render* and *generate animation* tasks to create the animation, as we observe in the final iteration.

Figure 3 shows that we can compose dynamic and complex workflow graphs by using our approach. Moreover, our approach has widespread applicability to other domains. The scenario of running multiple simulation instances to sample rare events is very common, e.g., in protein folding, self-assembled structures, and genetic algorithms.

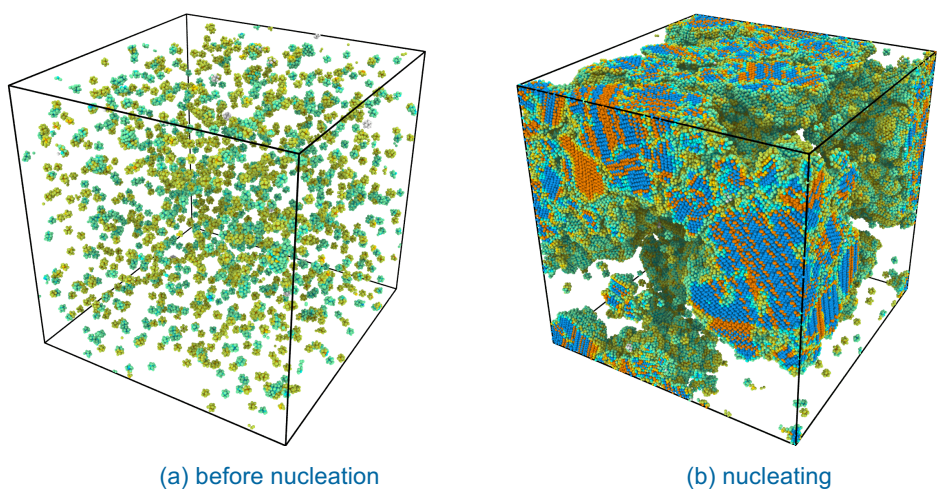


Figure 4. Stages of nucleation in freezing water system composed of 2,000,000 atoms

Capturing Nucleation In a Single Large Simulation Instance

We ran LAMMPS for 5,000,000 iterations and performed the diamond structure analysis every 10,000 iterations. Figure 4 shows the different phases of ice nucleation we were able to observe with our PyCOMPSs-Decaf model. This result demonstrates that our hierarchical workflow composition can enable the full science pipeline from the hypothesis of a nucleation event to the animation of a successful nucleation without any human interaction. In order to succeed, we used each system for what it does best. PyCOMPSs combines different workflow tasks (generating configuration files for different simulation instances, launching Decaf subworkflows, rendering) while Decaf accelerates the performance of this hierarchical workflow by performing the detection of diamond-shaped structures in situ alongside the molecular dynamics simulations.

Table I. Time and storage requirements for the different approaches when simulating the nucleation process. With a hierarchical workflow, we can fully automate the science path from hypothesis to discovery without requiring any human interaction. This also results in 64x storage savings.

Approach	Simulation time	Post-pro- cessing time	End-to-end exe- cution time	Storage re- quirements
PyCOMPSs- Decaf	11 hours	-	11 hours	250 MB
Traditional (post hoc)	11 hours	17+ hours	28+ hours	16 GB

Our approach has also several benefits compared with the traditional post hoc detection of diamond structures. The first is time savings, as shown in Table I. Normally, scientists have to perform the post-processing manually (i.e., diamond structure identification by using OVITO visualization software) for each of the 500 snapshots generated by the simulation. Our results indicate that this takes 12 seconds on average per snapshot, or a total of approximately 17 hours of post-processing time. This is a conservative estimate that does not include the extra time required to transfer files to the post-processing environment, reading files in OVITO, and user interaction time. We hide this post-processing time by detecting diamond-shaped structures in situ alongside LAMMPS.

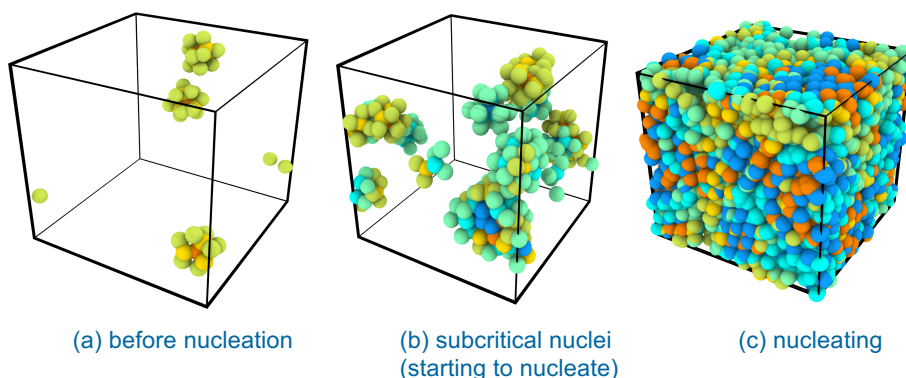


Figure 5. Stages of nucleation in freezing water system composed of 4,000 atoms

The second benefit is space savings, as shown in Table I. We reduced the storage requirements by only storing the positions of the nucleated atoms in the LAMMPS output files and assembling those into an animation, which only requires 250 MB storage space. On the other hand, the traditional approach saves all of the 500 output files (32 MB each) generated by LAMMPS for further post-processing. This occupies 16 GB of storage, which is 64 times higher than the PyCOMPSs-Decaf model.

Although we observed nucleation with a single large simulation, this problem size requires massive computing power, prohibiting most scientists from conducting such computations. For example, this experiment used 22,780 core-hours on the MareNostrum supercomputer. Next, we use fewer water molecules, which is more difficult to nucleate.

Capturing Nucleation by Running Many Instances of Small Simulations

As shown in Figure 3, we launched 60 instances of a Decaf subworkflow at three different undercoolings (210K, 220K, and 230K) with PyCOMPSs. We ran LAMMPS for 5,000,000 iterations and performed the diamond structure analysis every 10,000 iterations. We were able to observe nucleation in only 1 out of the 60 simulation instances with 210K due to the stochastic nature of the nucleation process. Moreover, it gets much harder to capture the nucleation with higher temperatures. We discarded the snapshots generated from unsuccessful simulation instances and saved only the results from experiments that successfully nucleated. In addition to saving 98% of storage space, this also allows scientists to focus on the results that they need. Figure 5 shows the stages of this nucleation in a freezing water system composed of 4,000 atoms.

This demonstrates that the PyCOMPSs-Decaf model supports the automated end-to-end science path for a complex science problem, which would otherwise require significant effort from scientists. The dynamic nature of PyCOMPSs, which afforded flexible control of running multiple simulation instances, and in situ coupling of the diamond detector to LAMMPS using Decaf were the keys to being able to capture rare events in an ensemble of simulations.

Table II. Number of processes and core-hours used by each simulation problem on MareNostrum. With hierarchical workflow, we can simulate the nucleation process by using 94% less computing resources.

Simulation size	Processes	Wall-clock time	Core-hours
Large (2M)	2,016	11 hours	22,780 hours
Small (4K)	480	2.5 hours	1,210 hours

Moreover, our hierarchical workflow composition achieved similar results as large simulations by using much fewer resources. For instance, this experiment used only 1,210 core-hours on the MareNostrum supercomputer compared with the large simulation problem which used 22,780 core-hours (94% reduction in the required resources), as shown in Table II. This shows that our approach can help scientists who usually do not have an access to large computing resources conduct such experiments.

CONCLUSION

Our results reveal that heterogeneous workflow integration is advantageous to both distributed and in situ workflow systems. On one hand, we can accelerate the performance of distributed workflows by providing access to HPC dataflow, where information is exchanged through shared memory or HPC interconnect instead of files. On the other hand, this composition of workflows extends in situ workflows with dynamicity and different programming models such as bag-of-tasks and looping that are not available in the dataflow HPC programming model. Our solution also combines Python and C++ workflow tasks, which can lead to the easier implementation of multilevel workflows for science applications.

Several avenues remain open for future work. Currently, PyCOMPSs has a coarse-grained synchronization model, where tasks in the workflow graph can start their execution only after the tasks on which they depend finish their execution. In the short-term, we plan to extend PyCOMPSs in a way that it can support finer-grained synchronization. For instance, one could start rendering the individual output files as soon as they are produced by Decaf subworkflows instead of waiting for an entire simulation snapshot. Our long-term goal is to create a common interface for different workflows to communicate with each other. A standardized interface would allow the propagation of the required information for workflow composition (e.g., resource requirements, input and output file locations, executable information) without being specific to any one workflow tool.

ACKNOWLEDGMENTS

This work is a collaboration between Argonne National Laboratory and Barcelona Supercomputing Center within the Joint Laboratory for Extreme-Scale Computing. This research is supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, under contract number DE-AC02-06CH11357, program manager Laura Biven, and by the Spanish Government (SEV2015-0493), by the Spanish Ministry of Science and Innovation (contract TIN2015-65316-P), by Generalitat de Catalunya (contract 2014-SGR-1051).

REFERENCES

1. J. Y. Choi *et al*, “Coupling exascale multiphysics applications: Methods and lessons learned,” in *Proceedings of IEEE International Conference on eScience 2018*, 2018.
2. T. Kuhlen *et al*, “Parallel in situ coupling of simulation with a fully featured visualization system,” in *Proceedings of the 11th*

- Eurographics Conference on Parallel Graphics and Visualization (EGPGV)*, 2011.
3. U. Ayachit *et al*, “Paraview catalyst: Enabling in situ data analysis and visualization,” in *Proceedings of the First Workshop on In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization*. ACM, 2015, pp. 25–29.
 4. F. Zheng *et al*, “PreDatA – preparatory data analytics on peta-scale machines,” in *Proceedings of the IEEE International Symposium on Parallel Distributed Processing (IPDPS ’10)*, 2010.
 5. M. Dreher and T. Peterka, “Decaf: Decoupled dataflows for in situ high-performance workflows,” Argonne National Lab.(ANL), Argonne, IL (United States), Tech. Rep., 2017.
 6. D. Hull *et al*, “Taverna: a tool for building and running workflows of services,” *Nucleic acids research*, vol. 34, no. suppl 2, pp. W729–W732, 2006.
 7. E. Afgan *et al.*, “The galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2016 update,” *Nucleic acids research*, p. gkw343, 2016.
 8. E. Tejedor *et al*, “Pycompss: Parallel computational workflows in python,” *The International Journal of High Performance Computing Applications*, vol. 31, no. 1, pp. 66–82, 2017.
 9. E. Deelman *et al.*, “Pegasus, a workflow management system for science automation,” *Future Generation Computer Systems*, vol. 46, pp. 17–35, 2015.
 10. S. Habib *et al.*, “Hacc: Simulating sky surveys on state-of-the-art supercomputing architectures,” *New Astronomy*, vol. 42, pp. 49–65, 2016.
 11. S. Plimpton *et al*, “Lammps-large-scale atomic/molecular massively parallel simulator,” *Sandia National Laboratories*, vol. 18, p. 43, 2007.
 12. J. F. Lofstead *et al*, “Flexible IO and integration for scientific codes through the adaptable IO system (ADIOS),” in *Proceedings of the 6th international workshop on Challenges of large applications in distributed environments*, ser. CLADE ’08. New York, NY, USA: ACM, 2008.
 13. M. Dorier *et al*, “Damaris: addressing performance variability in data management for post-petascale simulations,” *ACM Transactions on Parallel Computing (TOPC)*, vol. 3, no. 3, p. 15, 2016.
 14. J. Yu and R. Buyya, “A taxonomy of workflow management systems for grid computing,” *Journal of Grid Computing*, vol. 3, no. 3-4, pp. 171–200, 2005.

15. E. Deelman *et al*, “Workflows and e-science: An overview of workflow system features and capabilities,” *Future Generation Computer Systems*, vol. 25, no. 5, pp. 528 – 540, 2009.
16. I. Altintas *et al*, “Kepler: an extensible system for design and execution of scientific workflows,” in *Scientific and Statistical Database Management, 2004. Proceedings. 16th International Conference on*. IEEE, 2004, pp. 423–424.
17. Oliver *et al.*, (2018). Cylc: A Workflow Engine for Cycling Systems. *Journal of Open Source Software*, 3(27), 737.
18. D.Manubens-Gil *et al*, “Seamless management of ensemble climate prediction experiments on hpc platforms,” in *High Performance Computing & Simulation (HPCS), 2016 International Conference on*. IEEE, 2016, pp. 895–900.
19. R. Graves *et al.*, “Cybershake: A physics-based seismic hazard model for southern california,” *Pure and Applied Geophysics*, vol. 168, no. 3-4, pp. 367–381, 2011.
20. A. Stukowski, “Visualization and analysis of atomistic simulation data with ovito—the open visualization tool,” *Modelling and Simulation in Materials Science and Engineering*, vol. 18, no. 1, p. 015012, 2009.

ABOUT THE AUTHORS

Orcun Yildiz is a postdoctoral researcher at Argonne National Laboratory (IL, USA) since February 2018. He received his Ph.D. degree in Computer Science from Ecole Normale Supérieure de Rennes (France) in December 2017. His research interests include scientific workflows, Big Data processing, I/O management and high performance computing. Contact him at oyildiz@anl.gov.

Jorge Ejarque holds a Ph.D. from the Technical University of Catalonia (UPC) in 2015 and he is a Researcher at Barcelona Supercomputing Center contributing in the design and development of tools and programming models for distributed computing. He has worked in several national and international R&D projects and was a member of the experts' board of the Spanish National Grid Initiative. His current research interests are parallel programming models for distributed platforms and energy-efficient execution of distributed applications. Contact him at jorge.ejarque@bsc.es.

Henry Chan is a postdoctoral researcher at the Center for Nanoscale Materials (CNM), Argonne National Laboratory. He received his Ph.D. in computational chemistry from the University of Illinois at Chicago in 2015. He has extensive experience in simulations of soft-matter material systems and has published in high impact scientific

journals. His current research focus is on the application of machine learning principles in the development of interatomic force fields and techniques that enable accurate large-scale molecular simulations. Contact him at hchan@anl.gov.

Subramanian Sankaranarayanan is a scientist in the Nanoscale Science and Technology (NST) Division at Argonne and a Senior Scientist at the Institute of Molecular Engineering at University of Chicago. He obtained his Ph.D. in Chemical Engineering from University of South Florida in 2007. Prior to joining Argonne, Subramanian was a postdoctoral fellow at the School of Engineering and Applied Sciences at Harvard University from 2007-2010. His research at Argonne focuses on the use of machine learning to bridge the electronic, atomistic and mesoscopic scales. He uses supervised machine learning techniques to develop first principles based force fields for simulating reactive and mesoscopic systems. Other interests include machine learnt computational tool development for integrated X-ray imaging of ultrafast energy transport across solid-solid and solid-liquid interfaces. His interests span a diverse range of applications from tribology and corrosion to neuromorphic computing and thermal management. Contact him at ssankaranarayanan@anl.gov.

Rosa M. Badia holds a Ph.D. in Computer Science (1994) from the Technical University of Catalonia (UPC). She is a researcher from the Spanish National Research Council (CSIC) and team leader of the Workflows and Distributed Computing research group at the Barcelona Supercomputing Center (BSC). She was involved in teaching and research activities at the UPC from 1989 to 2008, where she was an Associate Professor since year 1997. Her current research interest are programming models for complex platforms (from multicore, GPUs to Grid/Cloud). Dr. Badia has published more than 150 papers in international conferences and journals in the topics of her research and he has participated in several international R&D projects. Contact her at rosa.m.badia@bsc.es.

Tom Peterka is a computer scientist at Argonne National Laboratory, scientist at the University of Chicago Consortium for Advanced Science and Engineering (CASE), adjunct assistant professor at the University of Illinois at Chicago, and fellow of the Northwestern Argonne Institute for Science and Engineering (NAISE). His research interests are large-scale parallel in situ analysis of scientific data. Recipient of the 2017 DOE early career award and three best paper awards, Peterka has published in ACM SIGGRAPH, IEEE VR, IEEE TVCG, and ACM/IEEE SC, among other top conferences and journals. Peterka received his Ph.D. in computer science from the

University of Illinois at Chicago in 2007, and he currently works actively in several DOE- and NSF-funded projects. Contact him at tpeterka@anl.gov.